Name: NGO TUNG CHUONG

ID: s225586848

GitHub: https://github.com/NoXxChapter/SIT221-DataStructure.git

- **Part A**

Tim and Te are two students studying algorithms and data structures. They are working on a web-system that allows selection of a time interval in history to obtain a daily maximum air temperature during the specified interval. Currently, Te stores the information about the temperature in a list sorted chronologically. Thus, if there are n days, then $t_1, \dots, t_n$ represents the list of temperatures, each associated with a particular date. A user's query consists of a pair (i, j) with $1 \le i < j \le n$, and the system is supposed to return $\max(t_i, \dots, t_j)$. Assume for simplicity that n is a power of 2, i.e., $n = 2^k$ for $k = 1, 2, 3, \dots$.

Tim believes that they should minimize the amount of computation per query since there will likely be a lot of traffic incoming to the system. His **first solution** is to precompute and store $\max(t_i, \dots, t_j)$ for every possible input (i, j). By doing so, he can respond to users' queries quicker. However, his concern is that such a solution needs storage of all pairs (values); thus, it requires a bit too much memory.

To decrease the memory consumption, Tim's **second solution** allows a small amount of computation per query. His idea is, rather than just store the $\max(t_i, \dots, t_j)$ for every (i, j), calculate and record values precomputed in such a way that, for any user's query, the server can retrieve two precomputed values and take the maximum of the two to return the final answer. Specifically, for a given list $t_1, \dots, t_n$, he proposes to split it into two sub-lists $t_1, \dots, t_{\frac{n}{2}}$ and $t_{\frac{n}{2}+1}, \dots, t_n$, and record

- $\max\left(t_i, \dots, t_{\frac{n}{2}}\right)$ for each $1 \le i \le \frac{n}{2}$, and
- $\max\left(t_{\frac{n}{2}+1}, \dots, t_j\right)$ for each $\frac{n}{2} < j \le n$.

He then recursively splits the two lists into two equal parts and stops the recursion when the list's size becomes equal to 1.

Your task is to evaluate the (best- and worst-case) time and space complexity for all three solutions: Te's current approach, the first and the second solution proposed by Tim. You need to follow their ideas about the data structures and show how values need to be precomputed and stored. Your answer can be in the form of a formal proof or a written explanation. You should make small numeric examples that will also help you to grasp the data structures.

For this answer, I will set a list of temperature for 4 days following: [10, 25, 15 ,20]

1st case – Te's solution:

This is a like a brute force way; Te keeps the temperature list chronologically. This is Te's data:

$$[10, 25, 15, 20]$$

So, for the runtime complexity, when the user wants to ask the maximum temperature between day 1 and day 4. The system must look at each element in the array. From 10, then 25, then 15 and then 20.

In worst case scenario, it could take to $O(n)$ time. As the bigger the data, the longer it takes for computer to do. The best possible case here is that $i = j$ (the range is only one day) then it takes only $O(1)$. And finally it takes up to $O(n)$ space to store all the data.

2nd case – Tim first's solution:

Precomputation method – 2D array method. With this Tim can minimize the time it takes to compare when he stores all the answer already for every possible (i, j) pair. And so, this is how Tim stores his data (I will still prefer the data to the prepare example)

	1	2	3	4
1	10	25	25	25
2	25	25	25	25
3	25	25	15	20
4	25	25	20	20

So, for now, if the user wants the highest temperature from day 1 to day 2. He only needs to access [1,2] → 25, it takes only $O(1)$. Because he simply looks up the answer in the table. But in contrast, since there are roughly n^2possible pairs of (i, j). The memory required grows very quickly as n increases, up to total $O(n^2)$.

3rd case – Tim second's solution:

The recursive split. Tim split the list into half $\left(1\ to \frac{n}{2}\ and \frac{n}{2} + 1\ to\ n\right)$. For the left half, Tim stores the max from every i up to the center. Meanwhile, for the right half, Tim stores the max from the center out to every j. He repeats it recursively until the list size is 1. With this Tim only take $O(1)$ time to look for the answer always take exactly two lookups and one comparison. But because the list is split in half recursively until the size is 1. Since $n = 2^k$here are $\log n$ levels of recursion. And at each level, Tim records values for n position. So, it takes up to $O(n \cdot logn)$ space.

So, for sample, the user wants the data from day 1 till day 3.

	1	2	3	4
	10	25	15	20
1st split (split between 2 and 3)	25	25	15	
2nd split (split between 1 and 2, 3 and 4)	10	25	15	
Final compare (2 and 3)	10	25	25	

- **Part B**

Based on your previous research about the Skip-List data structure, you should know that the insertion into a skip-list asks for flipping a coin until a head occurs. This rule determines the height for a tower representing a new element. So, for example, if we flipped two tails before a head, then we would insert an element of height 3. In all the examples you have seen before, the probability of flipping a head was exactly 1/2.

Now assume that the coin is biased, so the probability of flipping a tail on a given throw is 1/10. Derive an expression for the probability of producing an element of height h with this biased coin. Describe the consequence of this bias in terms of insertion into the skip-list and its structure.

So based on the biased given, we got the probability of TAIL throw if $\frac{1}{10}$. That mean, HEAD throw $= 1 - \frac{1}{10} = \frac{9}{10}$. Now we do a math; to reach each height we have to get those following results.

Height	Tail throw	Head throw	Probability
1	0	1	$P(H)$
2	1	1	$P(T) \cdot P(H)$
3	2	1	$P(T) \cdot P(T) \cdot P(H)$
....	...	...	...

n	$n-1$	1	$P(T)^{n-1} \cdot P(H)$

So, the expression for the probability of height is $\boldsymbol{P(h) = P(T)^{n-1} \cdot P(H)}$

So, if the coin has been biased like that it would give some consequences. With the insertion, it would likely cause the short tower effect which the coin will likely result in a Head on the very first flip most of the time. So very few elements will ever reach higher levels. In the end, the skip-list will look "flat" and very few nodes would make in through the upper express lanes.
In the performance, insertion and search most likely searching for an element will require scanning through nearly every node in the bottom layer. Since there are almost no "express lanes" to skip over nodes. Instead of the ideal $O(logn)$ search time, the performance will drift closer to $O(n)$, similar to a standard linked list. But the insertion process would be must faster cause the act of flipping the coin is faster (we stop sooner).

- **Part C**

After watching Lecture 6, you likely know that the maximum element of a binary max-heap can be obtained by means of the Max operation in a constant $O(1)$ time. However, there is an approach to find the kth maximum element in $O(k \log k)$ time, where $1 \leq k \leq n$. Keep in mind that both insertion and removal in a binary max-heap of size n takes a $O(\log n)$ time. Your task is to design such an algorithm and, as the answer, provide its pseudocode complemented by your explanation.

Input: A binary max-heap 'OriginalHeap' of size n, and an integer k
Output: The kth maximum element

Pseudocode FindKthMax(OriginalHeap, k):

```
If k > n or k <= 0: return Error

// Create a temporary heap to store (value, original_index)
tempHeap = new MaxHeap()

// insert the root of the original heap
tempHeap.insert(OriginalHeap[root].value, OriginalHeap[root].index)

For i from 1 to k-1:
  // get the current maximum from the temp list
  currentMax = tempHeap.extractMax()

  // find the children of this element in the original heap
  leftChild = getLeftChild(OriginalHeap, currentMax.index)
  rightChild = getRightChild(OriginalHeap, currentMax.index)

  // add the children as new candidates for the next largest value
  // we pick left first because leftChild > rightChild
  If leftChild exists:
    tempHeap.insert(leftChild.value, leftChild.index)
```

```
    If rightChild exists:
        tempHeap.insert(rightChild.value, rightChild.index)

  // the k-th element extracted is the answer
  Return tempHeap.extractMax().value
```

With this method, by utilize the temporary heap we can explore from the root the the kth node. And how it took $O(k \cdot logk)$ because first we perform k iterations. And in each iteration, we do 1 **extractMax** and up to 2 **insert operations** on the temp heap. The size of the temp heap never exceeds 2k (because we add 2 children but remove 1 parent each time). Since insertion and removal in a heap take $O(\log \mathrm{n})$, with n is size of heap by then each step takes $O(\log k)$. So total time is $\boldsymbol{k \cdot O(logk) = O(k \cdot logk)}$**.** And because we are **not modifying or removing elements from the original heap**. We only perform lookups $\mathrm{O}(1)$ and work within our smaller temp heap, which depends on k, not n. So, it **not an** $\boldsymbol{O(k \cdot logn)}$ this time.